ALGORITMOS CLÁSICOS Y ESTRUCTURAS DE DATOS

PUCMM
Pontificia Universidad Católica Madre y Maestra

TEMA:

Sistema de Gestión de Rutas de Transporte Público

TUTOR:

Freddy Peña

ALUMNOS:

Ángel Guzmán

Angel Belliard

IDs:

1016 – 0617

1016 – 1793

1. Resumen Ejecutivo

DoveGAMMA es un sistema de escritorio desarrollado en Java 17 con JavaFX que permite gestionar y optimizar rutas de transporte urbano. El sistema modela la red de transporte como un grafo dirigido ponderado, donde los nodos representan paradas y las aristas representan rutas con cuatro atributos de peso: tiempo, distancia, costo y transbordos.

El proyecto implementa tres algoritmos clásicos de optimización (Dijkstra, Bellman-Ford y BFS), una interfaz gráfica interactiva con SmartGraph, persistencia en MySQL, y un simulador de condiciones de tráfico en tiempo real con cuatro modos de transporte.

Características principales implementadas:

- Gestión CRUD completa de paradas y rutas con persistencia en MySQL.
- Cálculo de ruta óptima con Dijkstra (tiempo/distancia), Bellman-Ford (costo) y BFS (transbordos).
- Visualización interactiva del grafo con SmartGraph y resaltado de rutas.
- Cálculo de ruta alternativa bloqueando temporalmente el primer tramo.
- Simulador de condiciones: clima aleatorio (Soleado/Nublado/Lluvioso/Tormenta) + hora pico.
- Cuatro modos de transporte (Tren, Metro, Teleférico, Taxi) con factores de penalización.
- Popups animados con resumen de viaje (tiempo ajustado, distancia, costo, tramos).
- Panel de detalles por parada con rutas salientes y sus pesos.

2. Arquitectura y Estructura del Proyecto

El proyecto sigue una arquitectura en capas de dos niveles: la capa lógica (paquete **logica**) y la capa gráfica (paquete **grafica**). Ambas se comunican a través del patrón Singleton implementado en **AdaptadorVisual**, que actúa como puente entre la lógica del grafo y la interfaz de usuario.

2.1 Capa Lógica

Clase / Interfaz	Responsabilidad
GrafoTransporte	Núcleo del sistema. Implementa el grafo dirigido con HashMap de listas de adyacencia.
Parada	Vértice del grafo. Almacena id, nombre y coordenadas (x, y).
Ruta	Arista del grafo. Almacena tiempo, costo, distancia y transbordos.
DatoCamino	Wrapper para PriorityQueue en Dijkstra. Implementa Comparable.
CalculadorRuta	Fachada que selecciona el algoritmo según el criterio y lo ejecuta.
AlgoritmoRuta	Interfaz con métodos default: determinarPeso() y reconstruirCamino().
Dijkstra	Implementa AlgoritmoRuta. Optimiza tiempo y distancia.
BellmanFord	Implementa AlgoritmoRuta. Optimiza costo, soporta pesos negativos.
BFS	Implementa AlgoritmoRuta. Minimiza transbordos (saltos).
DFS	Implementa AlgoritmoRuta. Disponible en el código base.
VehiculoTransporte	Clase abstracta base para los 4 modos de transporte.
SimuladorCondiciones	Genera clima pseudoaleatorio y calcula factores de penalización.
GestorDB	Persistencia MySQL vía JDBC. CRUD completo de paradas y rutas.

2.2 Capa Gráfica

Clase / Archivo	Responsabilidad
AdaptadorVisual	Singleton. Conecta lógica y visual. Delega operaciones a GrafoTransporte y SmartGraph.
ControladorPrincipal	Controlador JavaFX. Maneja los formularios CRUD y las tablas de paradas y rutas.
PanelVisualizacion	StackPane con SmartGraphPanel. Maneja clicks, popups, animaciones y estilos.
FilaParada / FilaRuta	Modelos de fila para las TableView de JavaFX.
MainApp / Launcher	Punto de entrada de la aplicación JavaFX.
main.fxml	Declaración XML de la interfaz principal (BorderPane con sidebar y TabPane).
smartgraph.css	Hoja de estilos del grafo y de todos los componentes JavaFX.

3. Estructura de Datos: Grafo Dirigido

La estructura central del sistema es un **grafo dirigido ponderado** implementado mediante **listas de adyacencia** usando dos **HashMap**:

- **mapaParadas: HashMap<String, Parada>** — acceso O(1) a cualquier nodo por su ID.
- **listasAdyacencia: HashMap<String, List<Ruta>>** — por cada nodo, la lista de aristas salientes.

Esta elección es eficiente para redes de transporte dispersas donde la mayoría de las paradas no están conectadas entre sí directamente. Una matriz de adyacencia hubiera desperdiciado $O(V^2)$ de memoria innecesaria.

Atributos de una arista (Ruta):

Atributo	Tipo	Descripción
tiempo	double	Minutos estimados del tramo
distancia	double	Kilómetros del tramo
costo	double	Precio en pesos del pasaje
transbordo	int	Cantidad de transbordos requeridos

4. Análisis de Complejidad – Operaciones del Grafo

En las siguientes tablas, **|V|** representa el número de paradas (vértices) y **|E|** el número de rutas (aristas). Las complejidades se expresan en tres notaciones asintóticas según lo solicitado.

Operación	O (cota sup.)	Θ (cota ajust.)	Ω (cota inf.)
registrarParada()	O(1)	Θ(1)	Ω(1)
agregarRuta()	O(1)	Θ(1)	Ω(1)
eliminarParada()	O(|V| + |E|)	Θ(|V| + |E|)	Ω(|V|)
eliminarRuta()	O(grado_salida)	Θ(grado_salida)	Ω(1)
obtenerVecinos()	O(1)	Θ(1)	Ω(1)
existeArista()	O(grado_salida)	Θ(grado_salida/2)	Ω(1)
obtenerTodasLasRutas()	O(|V| + |E|)	Θ(|V| + |E|)	Ω(|V| + |E|)

Justificaciones:

registrarParada() / agregarRuta():

Ambas operaciones hacen una sola inserción en un HashMap, cuya operación put() es O(1) amortizado gracias al hashing. No recorren nada, solo ubican el bucket y asignan. Por eso las tres cotas son O(1), Θ(1), Ω(1).

eliminarParada():

Primero hace remove() en los dos HashMap del nodo — O(1) c/u. Luego itera sobre TODAS las listas de adyacencia de los demás nodos para borrar aristas entrantes, lo cual es O(|V| + |E|) en el peor caso (recorre todos los vecinos de todos los nodos). La cota inferior es Ω(|V|) porque al menos hay que revisar la lista de adyacencia de cada nodo existente.

eliminarRuta():

Llama a removeIf() sobre la lista de adyacencia del nodo origen. El peor caso es recorrer toda esa lista: O(grado_salida). El mejor caso es encontrar la ruta en el primer elemento: Ω(1).

obtenerVecinos():

Hace getOrDefault() en el HashMap — O(1) directo por hash. Retorna la referencia a la lista sin copiarla. Todas las cotas son O(1).

existeArista():

Recorre la lista de vecinos del origen buscando el destino. Peor caso: revisar toda la lista — O(grado_salida). Mejor caso: es el primer elemento — Ω(1). Caso promedio: Θ(grado_salida / 2).

5. Algoritmos Implementados

5.1 BFS — Breadth-First Search (Minimizar Transbordos)

Se usa cuando el criterio es **TRANSBORDOS**. BFS garantiza el camino con menor cantidad de saltos (aristas) porque explora el grafo nivel por nivel, usando una cola FIFO. Al no ponderar los pesos, trata todas las aristas como si costaran igual.

Estructura usada:

• Queue (LinkedList) para encolar nodos nivel a nivel. • HashSet de visitados para no procesar el mismo nodo dos veces. • HashMap paradasPrevias para reconstruir el camino al final.

Notación	Complejidad	Justificación
O (cota sup.)	O(|V| + |E|)	Cada nodo y cada arista se visita máximo una vez
Θ (cota ajust.)	Θ(|V| + |E|)	Siempre recorre todos los nodos y aristas alcanzables
Ω (cota inf.)	Ω(|V|)	Mínimo hay que tocar todos los vértices aunque no haya aristas

Complejidad espacial: *O(|V|) — la cola y el mapa de previos almacenan a lo sumo |V| elementos.*

5.2 DFS — Depth-First Search (Implementado, no activo en el selector)

DFS explora el grafo usando una pila (Stack), profundizando lo más posible en cada rama antes de retroceder. No garantiza el camino más corto ni el de menos saltos, pero es útil para detectar conectividad y ciclos.

Notación	Complejidad	Justificación
O (cota sup.)	O(|V| + |E|)	Igual que BFS — cada nodo y arista se visita máximo una vez
Θ (cota ajust.)	Θ(|V| + |E|)	Recorre todos los nodos y aristas alcanzables
Ω (cota inf.)	Ω(|V|)	Debe tocar al menos todos los vértices

5.3 Dijkstra (Minimizar Tiempo o Distancia)

Es el algoritmo principal para los criterios **TIEMPO** y **DISTANCIA**. Funciona con una **PriorityQueue** que siempre extrae el nodo con menor peso acumulado. Relaja los vecinos de ese nodo actualizando su distancia mínima si se encontró un camino mejor.

Restricción importante: Dijkstra solo funciona correctamente con pesos no negativos. Por eso no se usa para el criterio COSTO, donde podrían existir descuentos (valores negativos).

Estructura usada:

• PriorityQueue<DatoCamino> — cola de prioridad ordenada por peso acumulado. • HashMap distanciasMinimas — distancia mínima conocida a cada nodo. • HashMap paradasPrevias — de dónde vine para reconstruir el camino. • DatoCamino — wrapper con id + peso, implementa Comparable.

Notación	Complejidad	Justificación
O (cota sup.)	O((|V|+|E|) log |V|)	Cada extracción de la PriorityQueue cuesta O(log|V|), y se hacen a lo sumo O(|V|+|E|) operaciones
Θ (cota ajust.)	Θ((|V|+|E|) log |V|)	En la práctica siempre se procesan todos los nodos y todas las aristas alcanzables
Ω (cota inf.)	Ω(|V| log |V|)	Como mínimo hay que sacar cada nodo de la cola una vez

Complejidad espacial: *O(|V|) — los HashMap y la PriorityQueue almacenan como máximo |V| entradas.*

5.4 Bellman-Ford (Minimizar Costo)

Se usa para el criterio **COSTO** porque puede manejar pesos negativos (por ejemplo, descuentos). Relaja **|V| - 1** veces todas las aristas del grafo. Una iteración adicional detecta ciclos de peso negativo.

La relajación consiste en: para cada arista (u, v), si dist[u] + peso(u,v) < dist[v], actualizar dist[v] = dist[u] + peso(u,v).

Notación	Complejidad	Justificación
O (cota sup.)	O(|V| · |E|)	Se ejecutan |V|-1 rondas de relajación, cada una revisa todas las |E| aristas
Θ (cota ajust.)	Θ(|V| · |E|)	Siempre se completan las |V|-1 rondas sin optimización temprana
Ω (cota inf.)	Ω(|E|)	Mínimo hay que revisar cada arista al menos una vez

Complejidad espacial: *O(|V|) — HashMap distanciasMinimas y paradasPrevias con |V| entradas cada uno.*

6. Tabla Resumen de Complejidades

Algoritmo / Operación	Big-O (peor)	Theta (ajust.)	Omega (mejor)	Espacio
BFS	$O(\|V\|+\|E\|)$	$\Theta(\|V\|+\|E\|)$	$\Omega(\|V\|)$	$O(\|V\|)$
DFS	$O(\|V\|+\|E\|)$	$\Theta(\|V\|+\|E\|)$	$\Omega(\|V\|)$	$O(\|V\|)$
Dijkstra	$O((\|V\|+\|E\|)\log\|V\|)$	$\Theta((\|V\|+\|E\|)\log\|V\|)$	$\Omega(\|V\|\log\|V\|)$	$O(\|V\|)$
Bellman-Ford	$O(\|V\|\cdot\|E\|)$	$\Theta(\|V\|\cdot\|E\|)$	$\Omega(\|E\|)$	$O(\|V\|)$
registrarParada()	$O(1)$	$\Theta(1)$	$\Omega(1)$	$O(1)$
agregarRuta()	$O(1)$	$\Theta(1)$	$\Omega(1)$	$O(1)$
eliminarParada()	$O(\|V\|+\|E\|)$	$\Theta(\|V\|+\|E\|)$	$\Omega(\|V\|)$	$O(1)$
eliminarRuta()	O(grado_sal.)	Θ(grado_sal./2)	$\Omega(1)$	$O(1)$
obtenerVecinos()	$O(1)$	$\Theta(1)$	$\Omega(1)$	$O(1)$
existeArista()	O(grado_sal.)	Θ(grado_sal./2)	$\Omega(1)$	$O(1)$

7. Decisiones de Diseño

• ¿Por qué HashMap y no una matriz de adyacencia?

Una matriz de adyacencia requiere $O(|V|^2)$ espacio incluso si hay pocas conexiones. Para una red de transporte donde una parada normalmente conecta con 2-5 rutas, el HashMap con listas de adyacencia es mucho más eficiente en memoria: $O(|V| + |E|)$.

• ¿Por qué Dijkstra para tiempo y distancia?

Dijkstra es óptimo para grafos con pesos no negativos. Su complejidad $O((|V|+|E|)\log|V|)$ con PriorityQueue es mejor que Bellman-Ford $O(|V|\cdot|E|)$ cuando no hay pesos negativos. Los tiempos y distancias siempre son positivos.

• ¿Por qué Bellman-Ford para costo?

El costo puede tener descuentos (pesos negativos). Bellman-Ford es el único algoritmo de los tres que maneja correctamente este escenario, además de detectar ciclos negativos que invalidarían cualquier resultado.

• ¿Por qué BFS para transbordos?

Los transbordos se miden en cantidad de saltos (aristas), no en peso. BFS garantiza el mínimo número de saltos porque explora nivel por nivel. Usar Dijkstra aquí funcionaría igual, pero BFS es más simple y eficiente para este caso.

• Patrón Singleton en AdaptadorVisual

El AdaptadorVisual coordina tres capas: lógica del grafo, SmartGraph visual y base de datos MySQL. Un Singleton garantiza que todos usen la misma instancia y que los datos estén siempre sincronizados entre capas.

• Interfaz AlgoritmoRuta con métodos default

En lugar de repetir determinarPeso() y reconstruirCamino() en cada algoritmo, se implementaron como métodos default en la interfaz. Los algoritmos solo implementan su lógica específica de búsqueda, sin duplicar código.

• Ruta alternativa bloqueando el primer tramo

Para generar un 'Plan B', se elimina temporalmente la primera arista de la ruta óptima y se recalcula. Esto fuerza al algoritmo a encontrar un camino diferente. La arista se restaura inmediatamente después para no corromper el grafo.

8. Funcionalidades Implementadas

8.1 Gestión de Paradas y Rutas

El sistema permite agregar, modificar y eliminar tanto paradas como rutas mediante formularios en la interfaz gráfica. Cada operación se refleja simultáneamente en el grafo lógico, el grafo visual (SmartGraph) y la base de datos MySQL.

Operación	Descripción	Persistencia
Agregar Parada	Crea un nodo con ID, nombre y coordenadas X/Y	✔ MySQL
Modificar Parada	Actualiza el nombre de una parada existente	✔ MySQL
Eliminar Parada	Borra el nodo y todas sus rutas conectadas	✔ MySQL
Agregar Ruta	Crea arista dirigida con 4 pesos (tiempo, dist, costo, transbordo)	✔ MySQL
Modificar Ruta	Actualiza los pesos de una arista existente	✔ MySQL
Eliminar Ruta	Borra una arista específica entre dos paradas	✔ MySQL

8.2 Cálculo y Visualización de Rutas

El usuario hace doble click en una parada de origen en el grafo, luego doble click en la parada de destino. Aparece un menú de planificación donde selecciona:

- El **modo de transporte**: Tren, Metro, Teleférico o Taxi.
- El **criterio de optimización**: Tiempo, Distancia, Costo o Transbordos.

El sistema calcula la ruta óptima, la resalta en naranja en el grafo, y muestra un popup con animación del vehículo y tarjetas de resumen (tiempo ajustado por condiciones, distancia, costo y tramos). El usuario puede ver la ruta alternativa en azul con un botón.

8.3 Simulador de Condiciones en Tiempo Real

Al abrir el menú de planificación, se instancia un SimuladorCondiciones que genera un clima pseudoaleatorio basado en la hora del sistema y detecta si hay hora pico:

Condición	Descripción	Factor de impacto
Soleado	Clima despejado	+0% al tiempo
Nublado	Cielo cubierto	+5% base (multiplicado por factor lluvia del vehículo)
Lluvioso	Lluvia moderada	+20% base
Tormenta	Tormenta severa	+45% base — Teleférico suspendido
Hora Pico	7:00–9:30 / 17:00–19:30	Factor pico por vehículo (Tren +5%, Metro +15%, Taxi +50%)

9. Persistencia de Datos (MySQL)

La clase **GestorDB** maneja la conexión JDBC a una base de datos MySQL local (dovegamma). Al iniciar la aplicación, crea automáticamente las tablas si no existen y carga todos los datos previos. Al cerrar, todo ya está guardado porque cada operación CRUD llama al gestor inmediatamente.

Estructura de la base de datos:

Tabla paradas: id (PK), nombre, x, y

Tabla rutas: origen + destino (PK compuesta), tiempo, distancia, costo, transbordo

Ambas usan **INSERT ... ON DUPLICATE KEY UPDATE**, así que una operación de modificación se traduce automáticamente en un UPDATE sin necesidad de verificar si existe primero.

10. Pruebas Realizadas

Se realizaron pruebas funcionales manuales con una red de prueba de 7 paradas y 10 rutas para validar que cada algoritmo devuelve los resultados correctos.

10.1 Red de Prueba

ID	Nombre	Rutas salientes
P1	Centro	P2 (15min, 4km, $50, 0 transb.), P4 (25min, 8km, $80, 1 transb.)
P2	Norte	P3 (10min, 3km, $30, 0 transb.), P5 (20min, 6km, $60, 1 transb.)
P3	Aeropuerto	P6 (30min, 12km, $100, 0 transb.)
P4	Sur	P5 (8min, 2km, $25, 0 transb.)
P5	Este	P6 (12min, 4km, $40, 0 transb.), P7 (18min, 5km, $55, 1 transb.)
P6	Oeste	P7 (10min, 3km, $35, 0 transb.)
P7	Terminal	(destino final)

10.2 Casos de Prueba — Ruta P1 → P7

Criterio	Algoritmo	Ruta Resultante	Valor Total	¿Correcto?
Tiempo	Dijkstra	P1→P4→P5→P7	51 min	✔
Distancia	Dijkstra	P1→P4→P5→P7	15 km	✔
Costo	Bellman-Ford	P1→P4→P5→P7	$160	✔
Transbordos	BFS	P1→P2→P3→P6→P7	3 saltos	✔

10.3 Otras Pruebas

- **Nodo desconectado:** Se agregó P8 sin rutas. Los algoritmos retornaron lista vacía y la UI mostró 'No existe ruta'. ✔
- **Eliminar parada con rutas:** Se eliminó P5. El sistema eliminó correctamente P4→P5, P2→P5 y P5→P6, P5→P7. ✔
- **Ruta alternativa:** Se calculó la alternativa P1→P7. El sistema bloqueó P1→P4 y encontró P1→P2→P3→P6→P7. ✔
- **Persistencia:** Se cerraron y reabrieron varias veces. Todas las paradas y rutas cargaron correctamente desde MySQL. ✔
- **Tormenta + Teleférico:** Con clima Tormenta y modo Teleférico seleccionado, el sistema mostró el mensaje de servicio suspendido y bloqueó el cálculo. ✔

11. Conclusiones

• El sistema cumple con todos los requerimientos funcionales del proyecto: CRUD completo de paradas y rutas, cálculo de rutas con tres criterios diferentes, visualización interactiva del grafo, rutas alternativas, simulación de condiciones en tiempo real y persistencia en base de datos.

• La elección de listas de adyacencia con HashMap demostró ser la correcta para este tipo de red dispersa. Las operaciones de inserción y consulta de vecinos son O(1), lo cual es crítico para la interactividad de la interfaz.

• La interfaz AlgoritmoRuta con métodos default resultó ser una buena decisión de diseño: permitió agregar y cambiar algoritmos sin modificar el código del CalculadorRuta ni de la interfaz gráfica.

• El simulador de condiciones añade realismo al sistema sin complicar innecesariamente la lógica del grafo: los factores se aplican únicamente en la capa de presentación, no modifican los pesos reales de las aristas.

• El sistema es escalable: agregar nuevas paradas, rutas o incluso un nuevo algoritmo requiere cambios mínimos y localizados gracias a la separación en capas.